PROJECT EXPLANATION · INTERVIEW GUIDE

MeetAI – Local-First AI Meeting & Productivity Assistant

Architecture · Step-by-step workflow · Tech stack · How it works

React TypeScript Vite Tailwind Node.js Express MongoDB JWT Playwright FFmpeg faster-whisper Ollama · Llama 3 embeddinggemma RAG ChromaDB Gemini fallback Nodemailer

October 2026 · Windows 11 · Runs fully on a student laptop

Contents

1. The Project in One Minute 3
2. Problem and Solution 3
3. System Architecture 4
4. Tech Stack — What and Why 4
5. End-to-End Workflow (Step by Step) 6
6. Deep Dive: How the Bot Joins a Meeting 7
7. Deep Dive: Recording and Transcription 8
8. Deep Dive: AI Service Manager (Local First, Cloud Fallback) 9
9. Deep Dive: RAG — Asking Questions About Past Meetings 10
10. Email, Activity Tracking and Logging 10
11. Data Model and Storage 12
12. Security and Privacy 12
13. Real Run: A Live Google Meet Meeting 13
14. Testing and Quality 14
15. Engineering Challenges I Solved (Interview Stories) 14
16. Likely Interview Questions — With Answers 15
17. Demo Script (5 minutes) 15
18. How to Run It 17
19. Limitations and Future Work 17

1. The Project in One Minute

MeetAI is a **free, local-first AI meeting assistant**. You add a Google Meet link and a time. MeetAI **joins the meeting by itself**, **records** it, **transcribes** it on your own laptop, uses a **local LLM** to write the summary, decisions and action items, makes every meeting **searchable by meaning** (RAG), and **emails you the summary**. Everything runs on your machine; the cloud (Gemini) is only a backup.

1 min 43 s	₹0	123	55
from join to summary email in a real meeting	running cost: all core parts free & local	automated tests passing	issues found & fixed

How to say it: "I built a meeting assistant that runs entirely on a laptop. It joins Google Meet with Playwright, records with FFmpeg, transcribes with Whisper, and analyses with a local Llama 3 model through Ollama. It also has retrieval-augmented search across all past meetings and falls back to Gemini only if the local model fails."

2. Problem and Solution

Problem	How MeetAI solves it
Notes from meetings get lost; nobody remembers who agreed to what	Automatic summary, decisions, action items with owner + deadline, saved as tasks
Existing assistants (Otter, Fireflies…) are paid and upload private audio to the cloud	Whisper + Llama 3 run locally; audio never leaves the laptop by default
You must remember to start recording	Scheduler joins 2 minutes before start and records automatically
Can't search old meetings ("what did we decide about MongoDB?")	RAG: embeddings + vector search + LLM answer with citations
Cloud APIs have limits and outages	AI Service Manager: Ollama → Gemini key 1 → Gemini key 2 with automatic failover
Student budget	Only free/open-source tools; runs on a normal Windows laptop

3. System Architecture

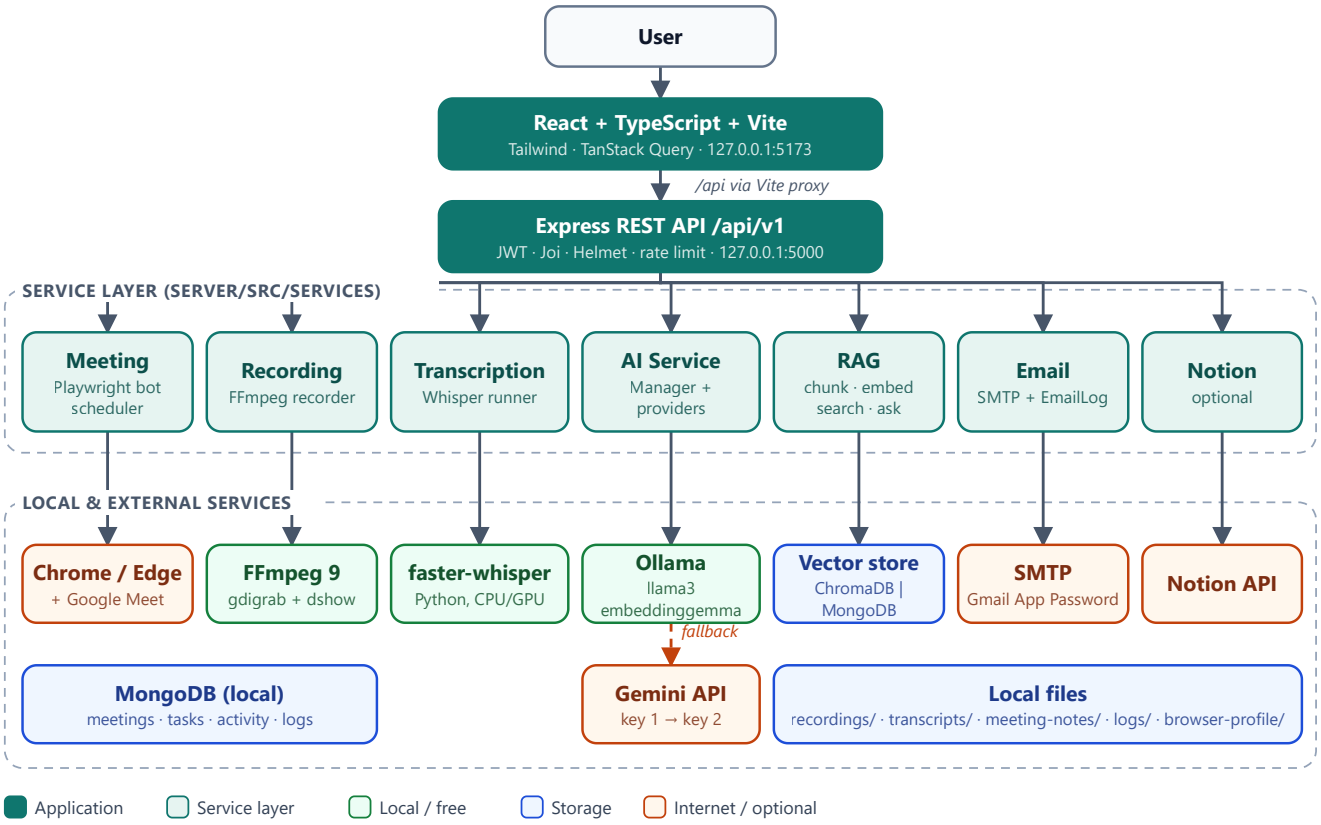


Figure 1: Overall architecture: three tiers + local AI services

MeetAI is a **three-tier application** that runs on one computer:

1. **Presentation tier**: React + TypeScript single-page app (Vite dev server on 127.0.0.1:5173). It only talks to the backend through `/api/v1`.
2. **Application tier**: Node.js + Express REST API (127.0.0.1:5000), organised as *routes* → *middleware* → *controllers* → *services* → *models*. Each capability is a **service**: Meeting bot, Recorder, Transcription, AI Service Manager, RAG, Email, Notion.
3. **Data tier**: MongoDB for structured data (meetings, tasks, activity, vectors) + the local disk for large files (MP4 recordings, transcripts, Markdown notes).

The services drive **local tools**: Chrome (via Playwright), FFmpeg, Python faster-whisper, and Ollama (Llama 3 + embeddinggemma). Internet services (Google Meet, Gemini, Gmail SMTP, Notion) are used only where unavoidable or optional.

> **How to say it:** "It's a layered monolith: a thin controller layer and a service layer that wraps each external tool, so I can swap Whisper implementations or AI providers without touching the API."

4. Tech Stack — What and Why

Layer	Technology	What it does in MeetAI	Why I chose it
Frontend	React 18 + TypeScript	Dashboard, meeting pages, Ask meetings, Settings	Component model + type safety

Layer	Technology	What it does in MeetAI	Why I chose it
Build	Vite 6	Dev server, proxy to API, production bundle	Very fast HMR; simple proxy
Styling	Tailwind CSS	All UI styling, dark mode	Fast to build consistent UI
Server state	TanStack Query	Fetching + polling job progress every 4 s	Caching and polling without Redux
Backend	Node.js + Express	REST API, background jobs, scheduler	Same language as frontend; huge ecosystem
Database	MongoDB + Mongoose	Meetings, tasks, activity log, email log, vectors	Flexible nested documents (AI output varies)
Auth	JWT + bcrypt	Optional multi-user login; refresh tokens	Stateless, industry standard
Validation	Joi	Validates every request AND the AI's JSON output	Declarative schemas, good errors
Security	Helmet, CORS, rate-limit, HPP, mongo-sanitize	HTTP hardening	Standard Express hardening set
Automation	Playwright + Chrome	Opens Meet, types name, clicks Join, detects end	Reliable locators, real Chrome profile
Media	FFmpeg	Screen + audio recording; WAV extraction	Free, universal, scriptable
Speech-to-text	faster-whisper (Python)	Local transcription	4× faster than original Whisper on CPU
LLM	Ollama + Llama 3 (8B)	Summary, decisions, action items, RAG answers	Free, private, runs offline
Embeddings	embeddinggemma (Ollama)	768-d vectors for search	Small (300M), local, good quality
Vector store	ChromaDB or MongoDB	Semantic search	Chroma when available; zero-setup Mongo fallback
Cloud fallback	Gemini API (2 keys)	Backup LLM when Ollama fails	Generous free tier
Email	Nodemailer + Gmail SMTP	Meeting summary emails	Free with an App Password
Logging	Winston	app / error / ai / meeting logs, secrets redacted	Structured JSON logs
Testing	Vitest + Supertest	123 automated tests	Fast, ESM-native

5. End-to-End Workflow (Step by Step)

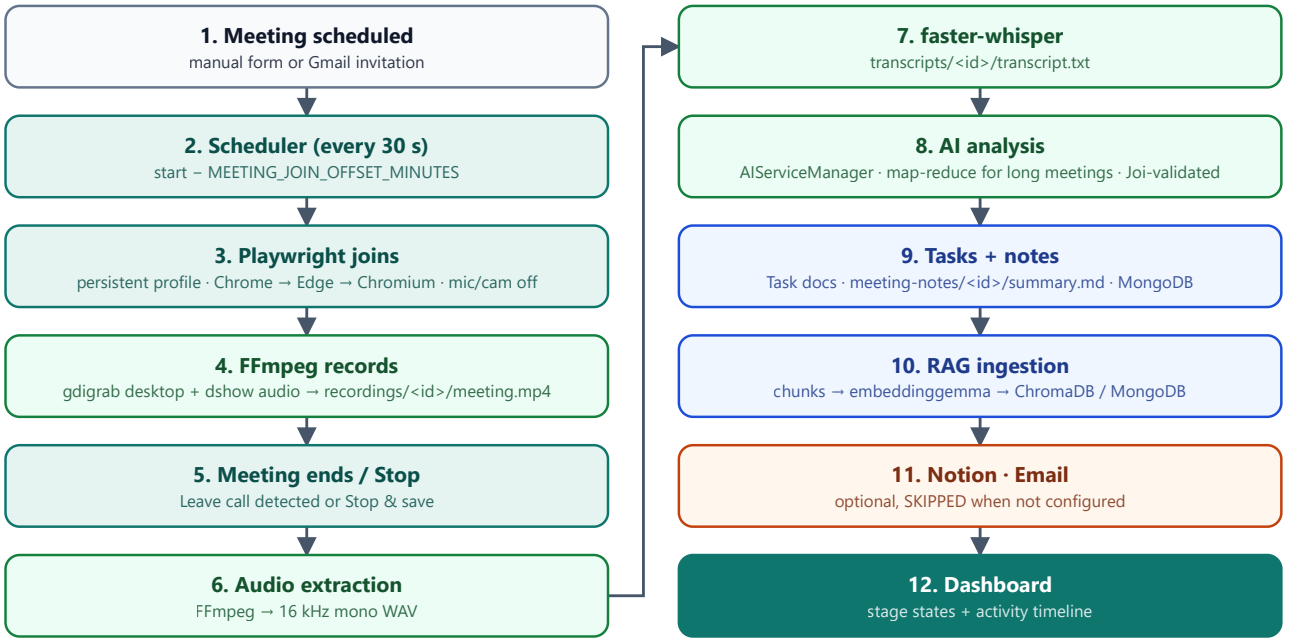


Figure 2: The 12-stage meeting pipeline

1. **Add a meeting** — title, Meet link and start time (manual form; Gmail invitation detection is optional).
2. **Scheduler** — every 30 s it looks for meetings starting within 2 minutes.
3. **Join** — Playwright opens Chrome with the saved profile, waits for Meet's join screen, clears pop-ups, types the name, keeps mic/camera off and clicks *Join now* / *Ask to join* (details in section 6).
4. **Record** — once inside the call, an FFmpeg process captures the screen and the audio device into `recordings/<id>/meeting-<time>.mp4`.
5. **Detect the end** — every 20 s the bot checks whether the call is still running; when it ends (or you press Stop) FFmpeg finalises the MP4 and the browser closes.
6. **Queue processing** — a background job is created (the API returns immediately; the UI polls progress).
7. **Audio extraction** — FFmpeg converts the MP4 to a 16 kHz mono WAV.
8. **Transcription** — faster-whisper turns speech into `transcript.txt` locally.
9. **AI analysis** — the AI Service Manager sends the transcript to Llama 3 (or Gemini if needed); the JSON reply is validated and normalised.
10. **Save results** — summary/decisions/action items stored in MongoDB, Markdown notes on disk, action items become Tasks.
11. **RAG ingestion** — notes + transcript are chunked, embedded with embeddinggemma and stored in the vector store.
12. **Notify** — Notion page (optional) and summary email; the dashboard timeline shows every step.

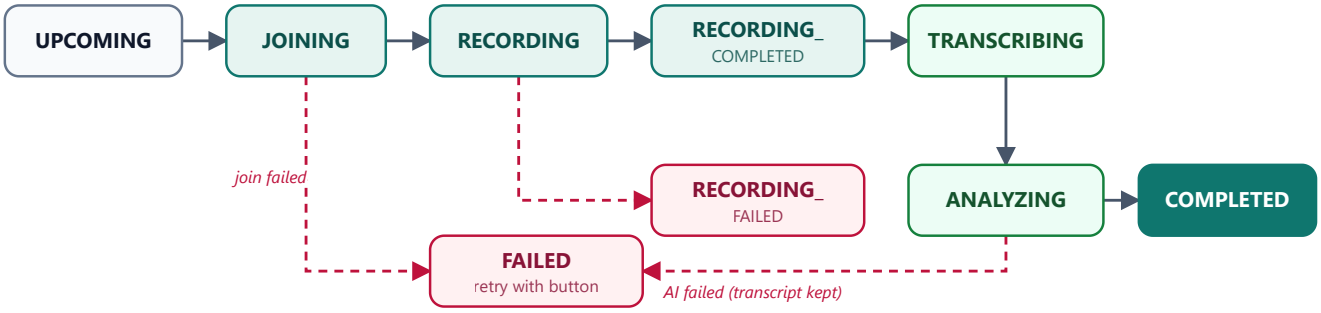


Figure 3: Meeting status lifecycle (a failure never deletes earlier files)

6. Deep Dive: How the Bot Joins a Meeting

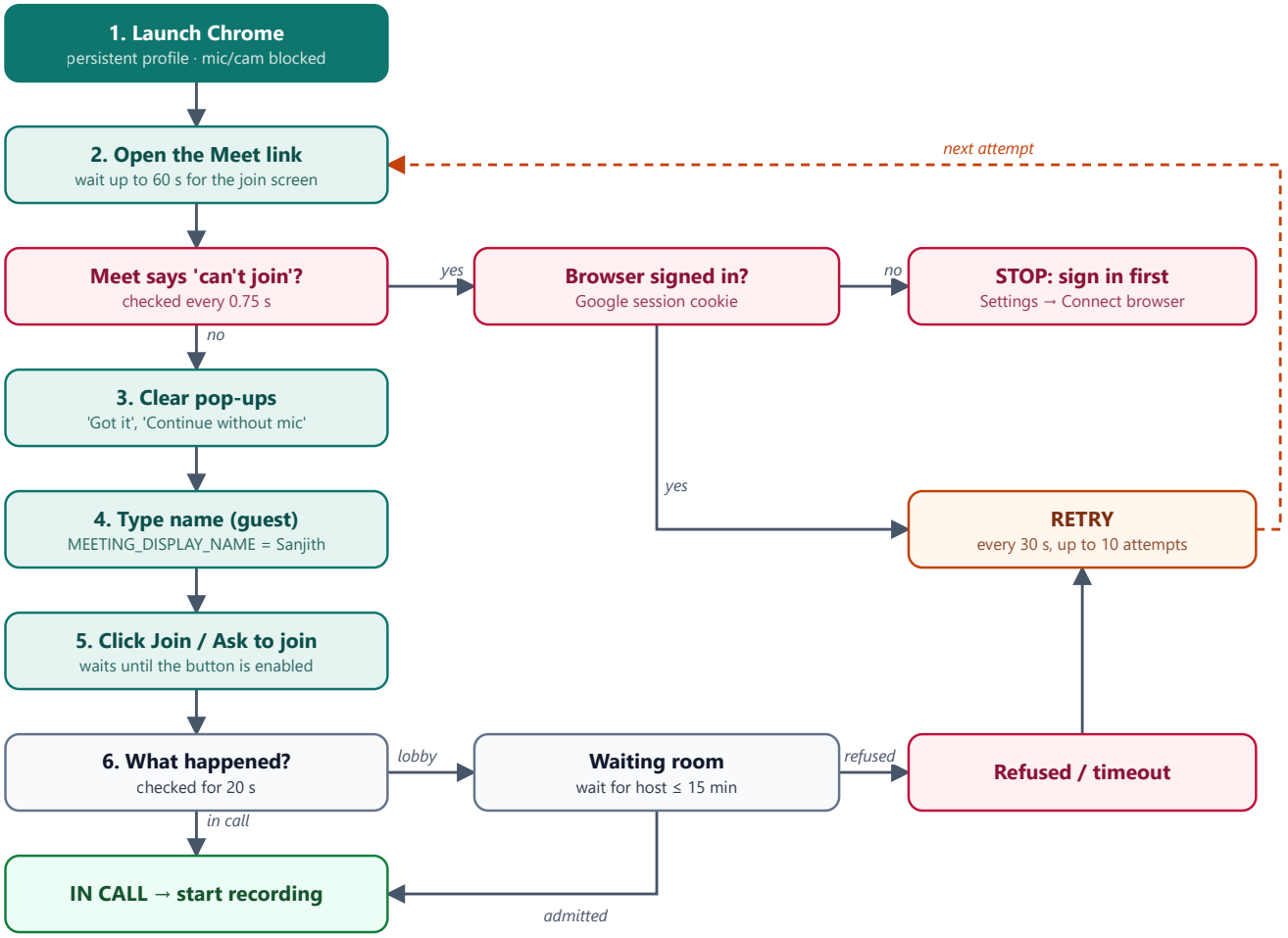


Figure 4: Join decision flow with retries and waiting-room handling

- **Signed-in profile:** you sign in to Google once in a normal Chrome window (Settings → Connect browser session). Google stores a login cookie in `browser-profile/`; MeetAI never sees the password. Later joins reuse that session.
- **Waiting for Meet to load:** Meet is a heavy single-page app — the Join button appears seconds after the page loads, so the bot polls for up to 60 s instead of checking once.
- **Guest screen:** if not signed in, Meet asks "What's your name?" and keeps Join disabled until a name is typed — the bot types the configured name and waits for the button to enable.

- **Mic and camera are OFF by design:** the browser is launched with no camera/mic permission (`--deny-permission-prompts`), so the bot can never be heard or seen.
- **Waiting room:** after "Ask to join" it waits up to 15 minutes for the host; recording starts only after admission.
- **Retries:** refused or failed attempts retry every 30 s (10 attempts); each attempt appears on the Activity timeline, with a screenshot saved to `logs/` for diagnosis.

> **How to say it:** "The hardest part was making browser automation robust against a real, changing UI: waiting for elements properly, handling guest and waiting-room states, and making failures diagnosable with screenshots instead of silent errors."

7. Deep Dive: Recording and Transcription



Figure 5: Recording

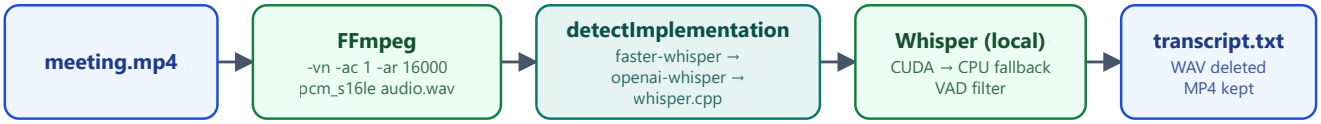


Figure 6: Transcription

- Playwright only drives the browser; **FFmpeg** is a separate process that records the desktop (`gdigrab`) and one audio device (`dshow`) to H.264 + AAC.
- Before recording it verifies the audio device exists (FFmpeg 9 lists devices but exits with code 0 — a bug I had to fix).
- Stopping sends `q` to FFmpeg so the MP4 is finalised properly (falls back to kill after 15 s).
- Transcription auto-detects faster-whisper → openai-whisper → whisper.cpp, falls back from GPU to CPU, and keeps the MP4 even if transcription fails.
- To capture other people's voices on Windows you need a loopback device (Stereo Mix or VB-Audio Cable); a microphone only hears the room.

8. Deep Dive: AI Service Manager (Local First, Cloud Fallback)

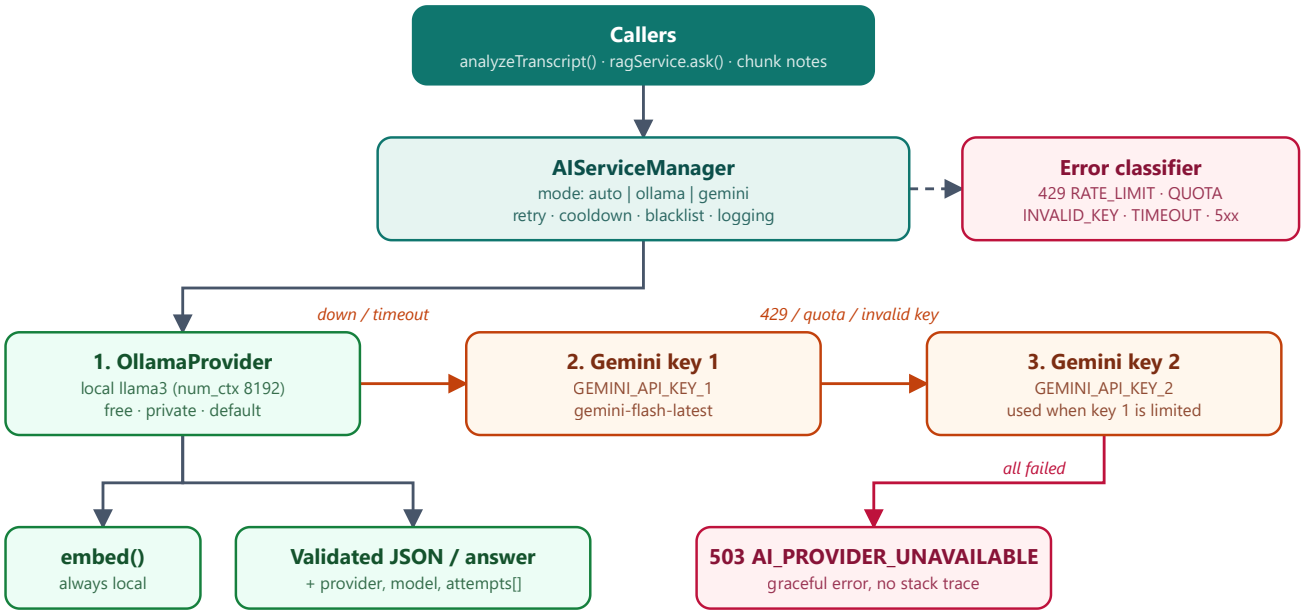


Figure 7: AIServiceManager: one place that decides which model answers

Failure	Detected by	What the manager does
Ollama not running	connection refused	move to Gemini key 1 (15 s cooldown on Ollama)
Timeout / 5xx	AbortController / status	retry once on the same provider
Rate limit (429)	HTTP 429	cool that key down (Google's retry hint or 60 s) → next key
Quota exhausted	429 + 'quota'	cool down 1 hour → next key
Invalid key	400 'API key not valid' / 403	disable that key permanently — never retried
Everything failed	—	clean `503 AI_PROVIDER_UNAVAILABLE` error

- **Modes:** `auto` (Ollama first), `ollama` (100% offline), `gemini` (cloud first).
- **Reliable JSON from a small model:** strict prompt + `format:"json"` → extract JSON from the reply → validate with Joi → normalise (e.g. empty priority becomes MEDIUM) → one repair prompt if invalid.
- **Long meetings:** transcripts over 12,000 characters are summarised in parts (map) and then merged (reduce), so nothing is silently cut by the 8k context window.
- **Security:** API keys live only in the backend `.env`, go only in an HTTP header, and are redacted from logs.

> **How to say it:** "I used a chain-of-responsibility style manager with error classification, so a 429 cools a key down, an invalid key is blacklisted, and the user only sees an error if every provider fails."

9. Deep Dive: RAG — Asking Questions About Past Meetings

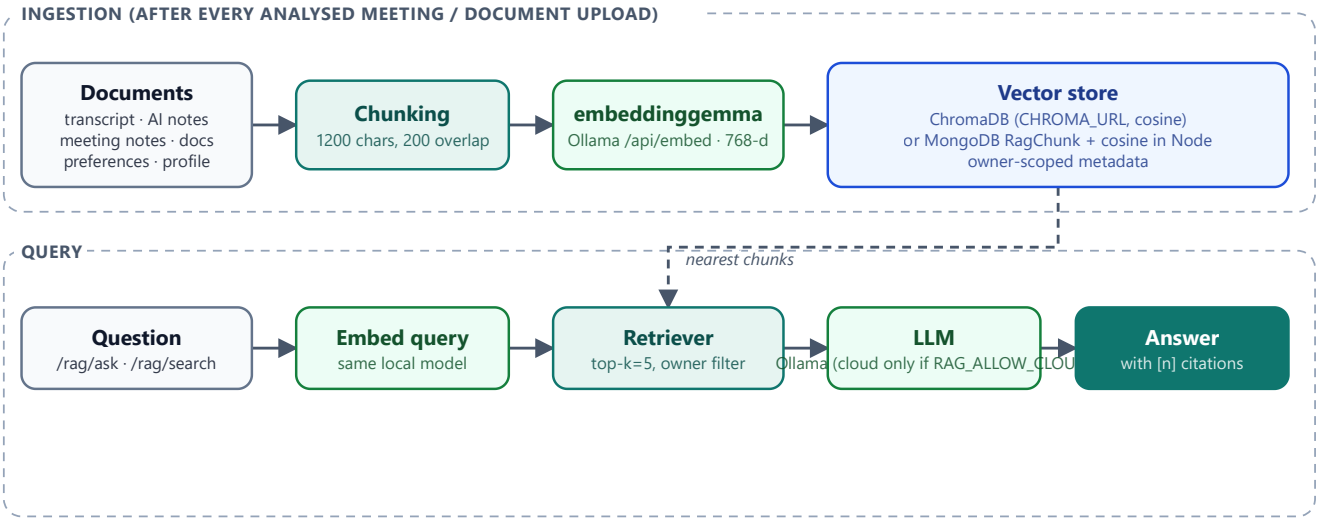


Figure 8: Retrieval-augmented generation

1. **Ingest:** after each meeting, the AI notes and transcript are split into ~1,200-character chunks with 200-character overlap (cut at sentence/word boundaries).
2. **Embed:** each chunk becomes a 768-number vector using `embeddinggemma` through Ollama (local).
3. **Store:** vectors go to ChromaDB if configured, otherwise to MongoDB — every chunk is tagged with its owner and meeting.
4. **Query:** the question is embedded with the same model; the 5 most similar chunks (cosine similarity) are retrieved.
5. **Answer:** Llama 3 answers *only* from those chunks and cites them as [1], [2]… Private context is never sent to the cloud unless explicitly allowed.

Example from a real test: *"What tasks were assigned in the Project Phoenix meeting?"* → "Rahul: create the database schema by October 15th; Anita: benchmark faster-whisper by Friday; Priya: write the project report draft by October 20th [2]".

10. Email, Activity Tracking and Logging

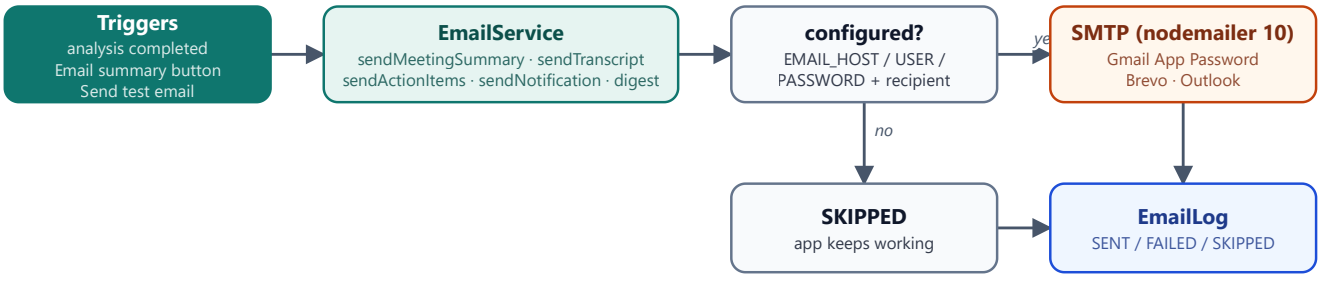


Figure 9: Email flow

- **Email:** Nodemailer over Gmail SMTP with an App Password. After analysis the summary (key points, decisions, action items, questions) is emailed to `EMAIL_TO`. Every attempt is stored in `EmailLog` as SENT / FAILED / SKIPPED; without SMTP the app keeps working.

- **Activity timeline:** every stage writes a row (PENDING / RUNNING / COMPLETED / FAILED / RETRYING / SKIPPED) — shown on the dashboard and in each meeting's Activity tab.
- **Logs:** Winston writes `meetai.log`, `error.log`, `ai.log` (provider choice, fallbacks) and `meeting.log` (pipeline). Passwords, tokens and keys are redacted automatically.

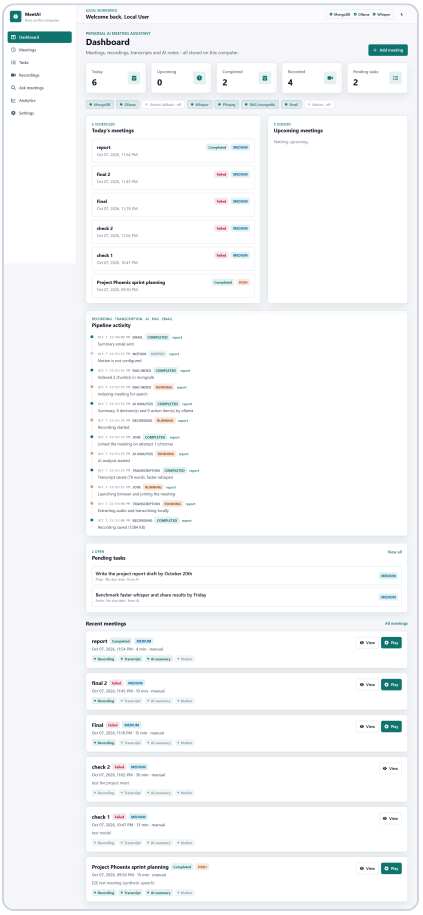


Figure 10: Dashboard: system status strip, live pipeline activity of a real meeting, tasks

11. Data Model and Storage

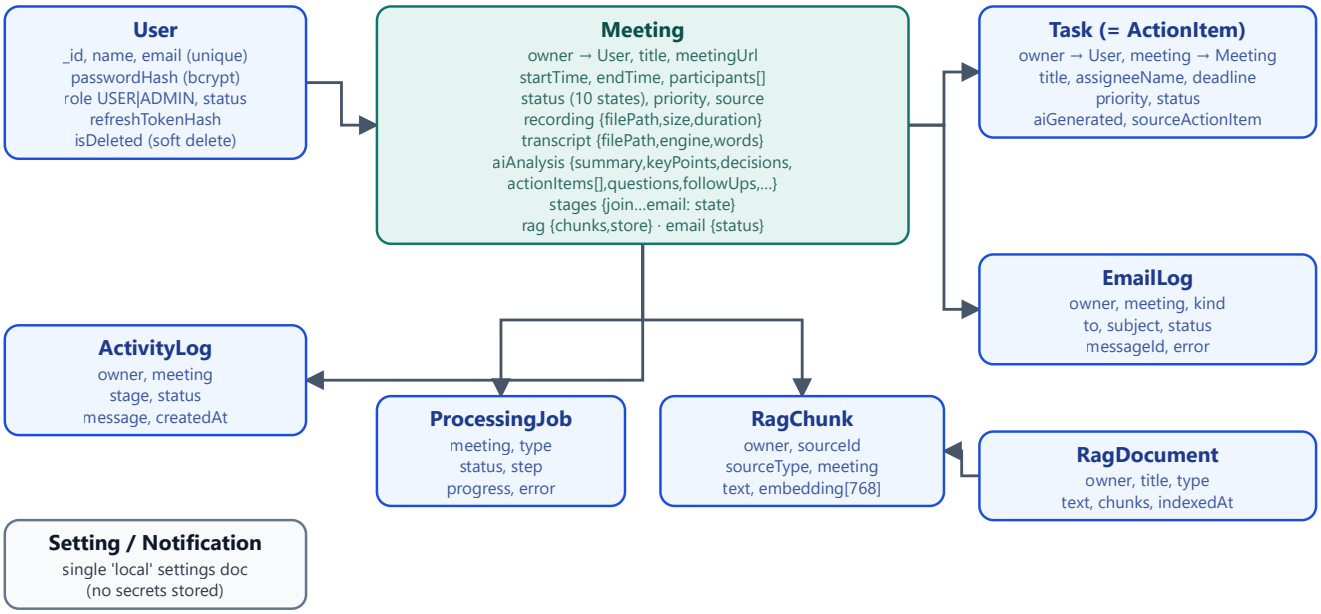


Figure 11: MongoDB collections and relations

- **MongoDB stores metadata only** — paths to files, never the MP4 itself.
- **Disk layout:** `recordings/<id>/meeting-<time>.mp4`, `transcripts/<id>/transcript.txt`, `meeting-notes/<id>/summary.md`, `browser-profile/`, `logs/`.
- **Data isolation:** every document has an `owner`; every query filters by it.
- **Soft delete** for meetings and tasks; deleting a meeting also removes its search vectors.

12. Security and Privacy

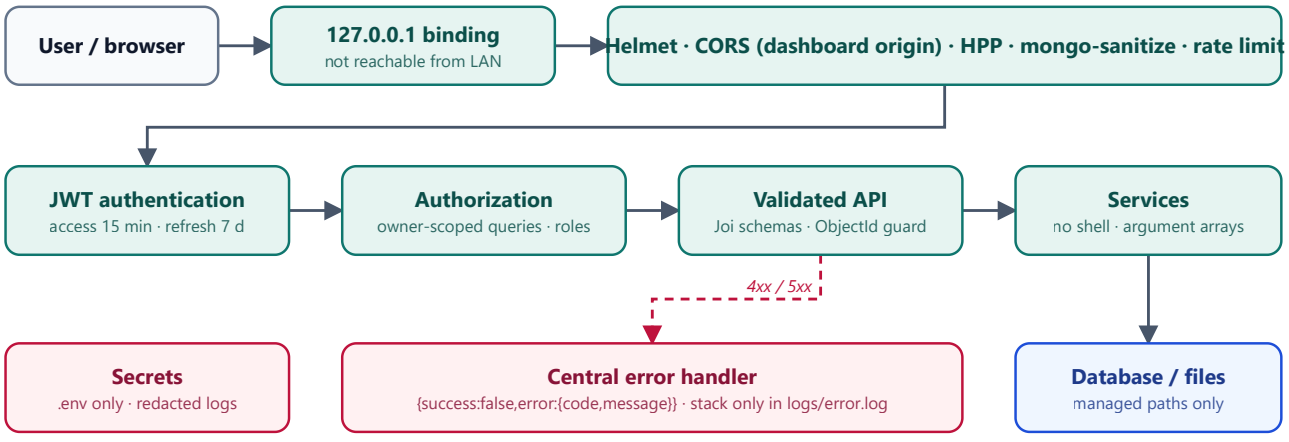


Figure 12: Security layers

Concern	Control
Network exposure	API and UI bind to 127.0.0.1 only — not reachable from Wi-Fi
Authentication	JWT access (15 min) + refresh (7 days, hashed, revoked on logout)
Passwords	bcrypt hashing; Google passwords are never handled by MeetAI
Input	Joi on every write; ObjectId guard; regex-escaped search; NoSQL operator stripping
Abuse	Rate limits (20 login attempts / 15 min)
Secrets	.env only (git-ignored), redacted from logs, never sent to the browser
Files	Only paths inside managed folders can be streamed (no path traversal)
Dependencies	npm audit: 0 production vulnerabilities

13. Real Run: A Live Google Meet Meeting

Measured from the activity log of a real meeting (“report”, 7 Oct 2026, Windows 11 laptop, CPU-only Whisper):

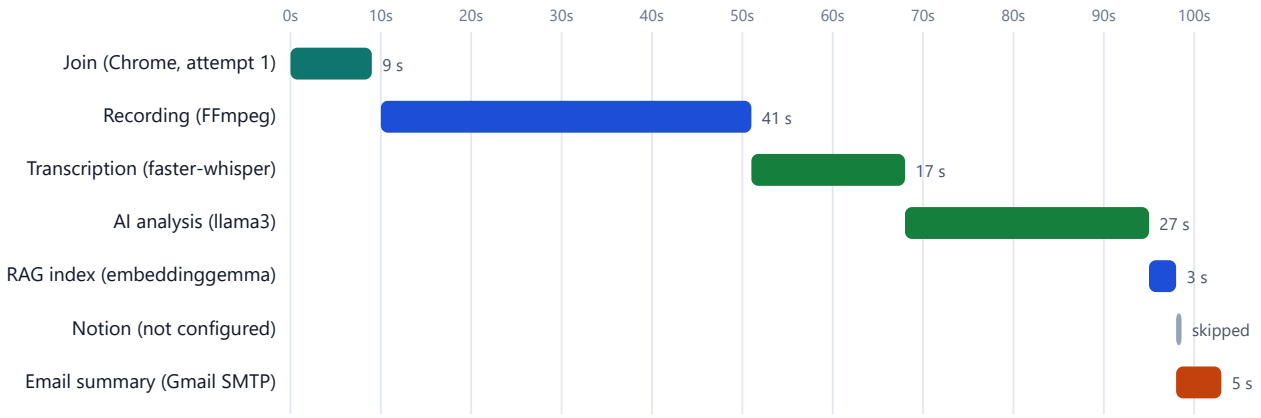


Figure 13: Real meeting timeline — from joining to the summary email

Step	Result
Join	Chrome joined on the first attempt in 9 s
Recording	40 s of video (41 s wall-clock), 1.3 MB MP4 (H.264 + AAC)
Transcription	79 words in 17 s with faster-whisper (base model, CPU)
AI analysis	27 s with Llama 3 via Ollama
RAG	2 chunks indexed in 3 s
Email	Summary email delivered via Gmail SMTP
Total	1 min 43 s from browser launch (18:22:17) to email sent (18:24:00)

Controlled end-to-end test (88 s of synthetic 3-speaker speech): transcription 19 s, analysis 37 s, whole job 58 s; 23/23 steps passed; all three action items extracted with owner and deadline.

14. Testing and Quality

Type	What	Result
Unit	AI JSON parsing, Gemini error classification, chunking, FFmpeg args, join labels	pass
Integration (real MongoDB)	API, auth, authorization, security, RAG, pipeline	pass
Mocked HTTP	Ollama + Gemini providers, key never leaked	pass
Live	ChromaDB server; real Gemini API with invalid keys	pass
Browser	Join flow vs. realistic Meet mock: slow load, guest screen, waiting room, refusal, retries	pass
End-to-end	Speech → MP4 → Whisper → Llama 3 → tasks → RAG → email	23/23
Static	TypeScript, ESLint, production build, npm audit	clean

123 automated tests pass (1 live ChromaDB test is skipped when no Chroma server is running).

15. Engineering Challenges I Solved (Interview Stories)

Challenge	Root cause	Fix
Bot said "no Join button" while it was on screen	Playwright's `isVisible({timeout})` ignores the timeout — it checked once before Meet rendered	Real waits/polling up to 60 s; verify outcome after clicking
Meet: "You can't join this video call"	Bot's browser profile was never signed in (verified from its cookie store)	Sign-in in a normal Chrome window; stop with instructions instead of retrying
Guest join timed out	Join stays disabled until a name is typed; a tooltip covered it	Dismiss pop-ups, type name, wait for enabled, fallbacks
No log file was ever written	Log formatter returned a new object, losing Winston's internal Symbol keys	Mutate the record in place
Whole AI summary rejected	Llama 3 returned an empty "priority" and strict validation failed everything	Tolerant normalisation for cosmetic fields
App crashed mid-meeting on Windows	Vite watched the recordings folder; the locked MP4 caused EBUSY	Exclude data folders from the watcher
Audio device list always empty	FFmpeg 9 exits 0 for device listing; code only parsed the error path	Parse both paths
API reachable from the Wi-Fi with no login	Servers bound to 0.0.0.0	Bind to 127.0.0.1 by default
Second join overwrote a recording being transcribed	Join button visible during processing; fixed file name	409 busy guard + timestamped files
Cloud fallback broken	Configured Gemini model had been shut down	`gemini-flash-latest` + key rotation + classification

> **How to say it:** Pick 2–3 of these and tell them as Situation → Root cause → Fix → Result. Interviewers value debugging stories with evidence.

16. Likely Interview Questions — With Answers

Question	Answer
Why local AI instead of OpenAI?	Zero cost, privacy (meeting audio is sensitive) and works offline. Gemini is only a fallback.
How do you get reliable JSON from a small LLM?	JSON mode, a strict schema in the prompt, extraction from the reply, Joi validation, normalisation, and one repair retry.
What is RAG and why use it?	Retrieve the most relevant chunks with embeddings, then let the LLM answer only from them — accurate, citable answers without fine-tuning.
Why MongoDB?	AI output is nested and evolves (key points, action items…); documents fit naturally. It also doubles as a vector store fallback.
How do long tasks not block the API?	The API returns 202 and a job id; a background queue runs one job at a time; the UI polls status.
How do you handle failures?	Each stage is isolated, transient errors retry, optional stages never fail a meeting, and earlier files are never deleted.
How do you keep API keys safe?	Backend .env only, header-only transport, log redaction, never in the React bundle.
How does the bot know the meeting ended?	It polls every 20 s for Meet's end screens and the Leave button, with a grace period and two consecutive misses to avoid false stops.
How did you test browser automation without real meetings?	A local mock that behaves like Meet (slow render, guest screen, waiting room, refusals) plus a real recording test.
What would you improve next?	Speaker diarisation, a persistent job queue, Teams/Zoom adapters and a packaged desktop installer.

17. Demo Script (5 minutes)

1. **Dashboard** — show the status strip (MongoDB, Ollama, Whisper, FFmpeg, RAG, Email all green) and explain local-first.
2. **Add meeting** — paste a Meet link and time; explain the scheduler joins 2 minutes early.
3. **Join & record** — Chrome opens and joins; open the **Activity** tab to show live stages and retries.
4. **Stop & save** — processing runs in the background; show the transcript and AI summary tabs.
5. **Tasks** — action items became tasks with owners.
6. **Ask meetings** — ask "What decisions were made?" and show the cited answer.
7. **Email** — show the summary email in the inbox.
8. **Settings** — explain AI modes (auto / local only / Gemini) and the join name.

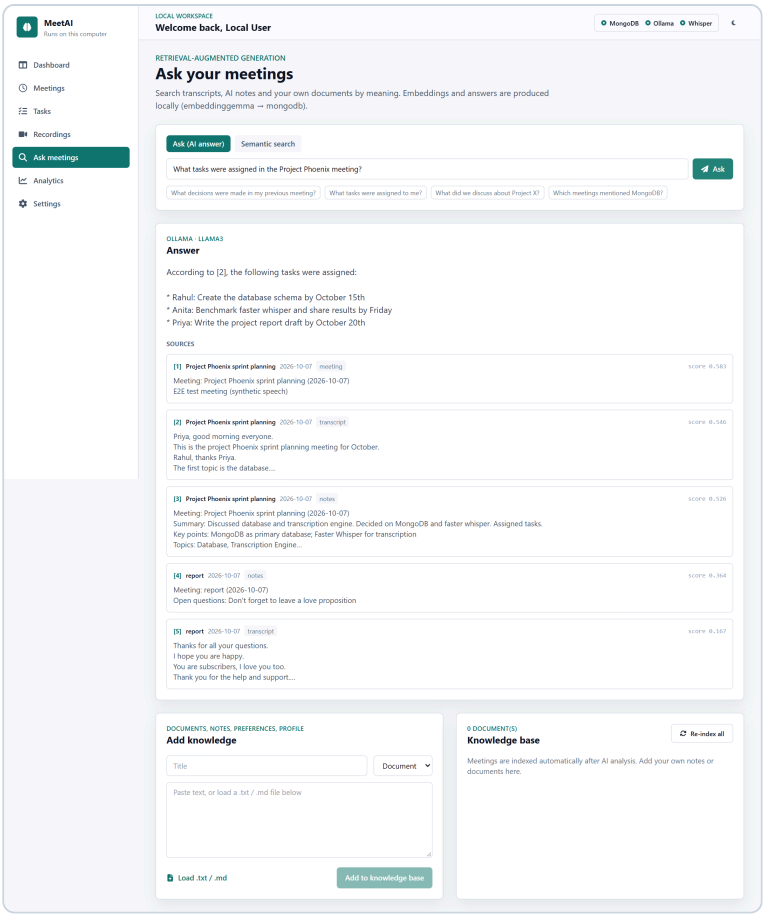


Figure 14: Ask meetings — RAG answer with cited sources

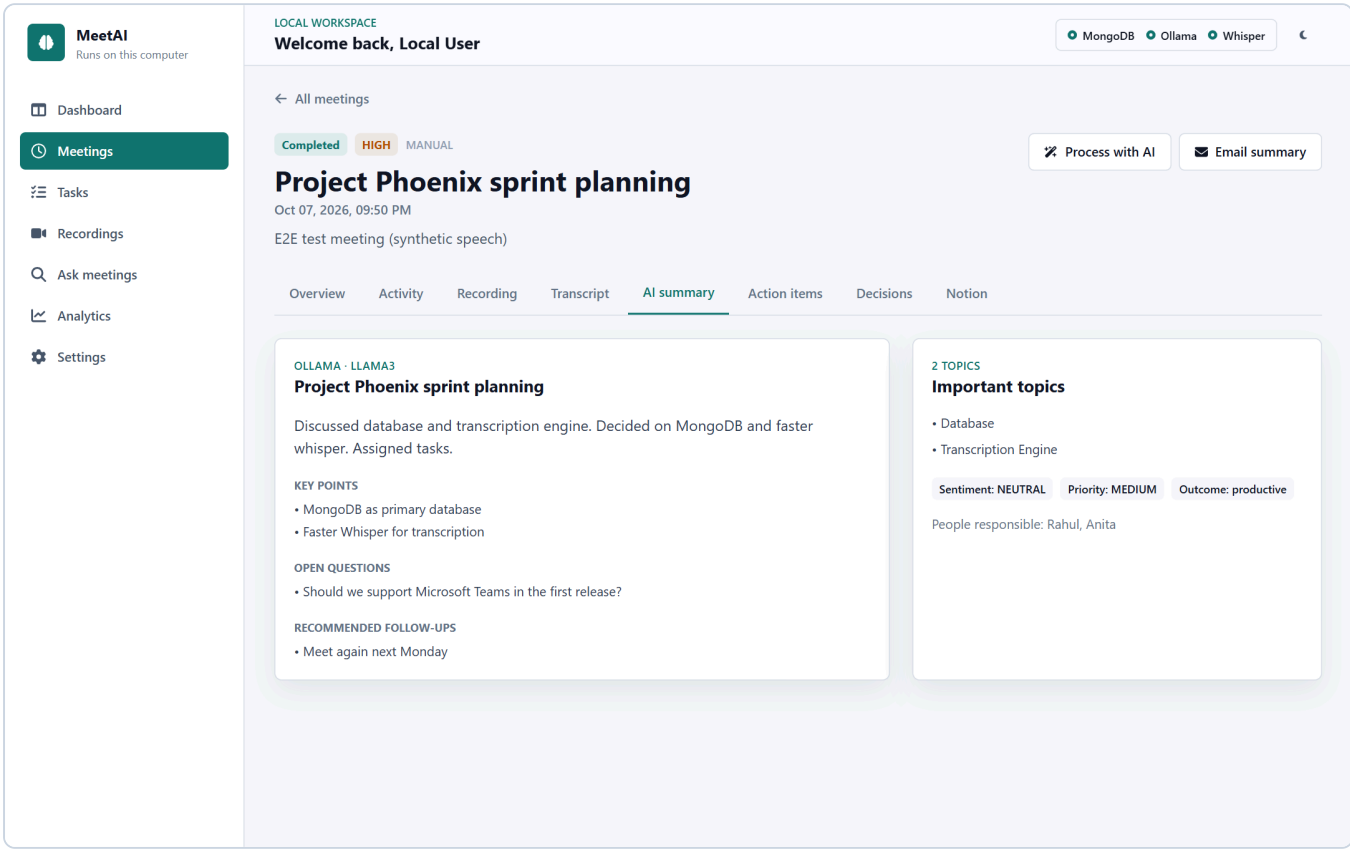


Figure 15: AI summary generated locally by Llama 3

18. How to Run It

```
# one-time
winget install OpenJS.NodeJS.LTS MongoDB.Server Ollama.Ollama Gyan.FFmpeg
pip install faster-whisper
ollama pull llama3
ollama pull embeddinggemma
npm install
Copy-Item .env.example .env

# every time
npm run dev            # UI http://localhost:5173  ·  API http://127.0.0.1:5000
npm test               # 123 tests
npm run check:local    # health of every dependency
```

- Sign the bot in once: Settings → Connect browser session.
- Email: put a Gmail App Password in `EMAIL_PASSWORD`.
- Meeting audio: enable Stereo Mix or install VB-Audio Virtual Cable and pick it in Settings.

19. Limitations and Future Work

Today	Next
Windows needs a loopback device to record other participants	Bundle a virtual audio driver guide / auto-detect
No speaker identification	Speaker diarisation (pyannote)
In-process job queue (lost on restart)	Persistent queue (Agenda on MongoDB)
Google Meet only	Zoom and Teams adapters
Meet UI changes can break selectors	Text-based detection + screenshot diagnostics already reduce this; add self-tests
Runs from a terminal	Desktop installer (Electron/Tauri)